ENV vars:

VITE_B2C_SPA_CLIENT_ID=c79bcc06-d6b3-4e30-80e2-af9790b60d9b
VITE_B2C_AUTH_DOMAIN=greenimpactcust.ciamlogin.com
VITE_B2C_TENANT=greenimpactcust.onmicrosoft.com
VITE_API_SCOPE=api://4bf050bc-5538-4395-bfce-7ce55febbec2/access_as_user
VITE_API_BASE=https://greentrust-api.delightfulwater-099c24c3.spaincentral.azurecontainerapps.io

Dependencias:

npm i @azure/msal-browser @azure/msal-react

src/authConfig.ts:

```
import type { Configuration } from "@azure/msal-browser";

const DOMAIN = import.meta.env.VITE_B2C_AUTH_DOMAIN;   // greenimpactcust.ciamlogin.com
const TENANT = import.meta.env.VITE_B2C_TENANT;        // greenimpactcust.onmicrosoft.com
const CLIENT = import.meta.env.VITE_B2C_SPA_CLIENT_ID; // GUID del SPA
const SCOPE  = import.meta.env.VITE_API_SCOPE;         // api://.../access_as_user

// En CIAM/B2C usa TU dominio + tenant y marca knownAuthorities:
const authority = `https://${DOMAIN}/${TENANT}/`;

export const msalConfig: Configuration = {
  auth: {
    clientId: CLIENT,
    authority,
    knownAuthorities: [DOMAIN],     // requerido para B2C/External ID
    redirectUri: window.location.origin,
    postLogoutRedirectUri: window.location.origin,
  },
  cache: { cacheLocation: "localStorage" },
};

export const loginRequest = { scopes: [SCOPE] };
```

src/msalClient.ts:

```
import { PublicClientApplication } from "@azure/msal-browser";
import { msalConfig, loginRequest } from "./authConfig";

export const msal = new PublicClientApplication(msalConfig);

export async function ensureSignedIn() {
  const accounts = msal.getAllAccounts();
  if (accounts.length === 0) {
    await msal.loginRedirect(loginRequest);
  }
}

export async function getAccessToken(): Promise<string> {
  const account = msal.getAllAccounts()[0];
  try {
    const r = await msal.acquireTokenSilent({ ...loginRequest, account });
    return r.accessToken;
  } catch {
    await msal.acquireTokenRedirect({ ...loginRequest, account });
    return ""; // tras el redirect, la app vuelve y repites la llamada
  }
}

export function logout() {
  return msal.logoutRedirect();
}
```

`src/api.ts` (llamada a tu back, este endpoint es el de autenticacion real de mi api, deberia funcionar tal cual si todo esta bien):

```
import { ensureSignedIn, getAccessToken } from "./msalClient";

const API_BASE = import.meta.env.VITE_API_BASE;

export async function callMe() {
  await ensureSignedIn();
  const token = await getAccessToken();
  const r = await fetch(`${API_BASE}/auth/me`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (!r.ok) throw new Error(`API error ${r.status}`);
  return r.json();
}
```

EJEMPLOS CHAT GPT (por si te ayudan en algo):

```
// src/main.tsx
import React from "react";
import ReactDOM from "react-dom/client";
import { MsalProvider } from "@azure/msal-react";
import App from "./App";
import { msal } from "./msalClient";

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <MsalProvider instance={msal}>
      <App />
    </MsalProvider>
  </React.StrictMode>
);
```

```
// src/App.tsx
import { useState } from "react";
import { callMe } from "./api";
import { msal, logout } from "./msalClient";

export default function App() {
  const [result, setResult] = useState<any>(null);

  return (
    <div style={{ fontFamily: "system-ui", padding: 24 }}>
      <h1>GreenImpact — Auth demo</h1>
      <div style={{ display: "flex", gap: 12 }}>
        <button onClick={() => msal.loginRedirect()}>Login</button>
        <button onClick={async () => setResult(await callMe())}>Llamar /auth/me</button>
        <button onClick={() => logout()}>Logout</button>
      </div>
      <pre style={{ marginTop: 24, background: "#f6f7f9", padding: 16 }}>
        {result ? JSON.stringify(result, null, 2) : "Resultado aquí..."}
      </pre>
    </div>
  );
}
```

MSAL debe instanciarse fuera del árbol de componentes y envolver la app con `MsalProvider`. El sample oficial lo hace así.

(Opcional) Proxy en Vite para evitar CORS en dev (no entiendo muy bien esto pero te lo dejo por si tu si):

```
// vite.config.ts
export default {
  server: {
    proxy: {
      "/api": { target: "http://localhost:5000", changeOrigin: true },
    },
  },
};
```

Y entonces llamas a `fetch("/api/auth/me")`. Doc oficial de Vite para `server.proxy`